should be a confidential client with credentials to authenticate to the AS.

- Client-side JavaScript applications that run in the browser, such as single-page apps. A client-side application is always a public client as it has no secure place to store a client secret.
- Mobile, desktop, and command-line applications. As for client-side applications, these should be public clients, because any secret embedded into the application can be extracted by a user.

In the authorization code grant, the client first redirects the user's web browser to the authorization endpoint at the AS, as shown in figure 7.5. The client includes its client ID and the scope it's requesting from the AS in this redirect. Set the `response_type` parameter in the query to `code` to request an authorization code (other settings such as `token` are used for the implicit grant). Finally, the client should generate a unique random `state` value for each request and store it locally (such as in a browser cookie). When the AS redirects back to the client with the authorization code it will include the same `state` parameter, and the client should check that it matches the original one sent on the request. This ensures that the code received by the client is the one it requested. Otherwise an attacker may be able to craft a link that calls the client's redirect endpoint directly with an authorization code obtained by the attacker. This attack is like the Login CSRF attacks discussed in chapter 4, and the state parameter plays a similar role to an anti-CSRF token in that case. Finally, the client should include the URI that it wants the AS to redirect to with the authorization code. Typically,